



SAPIENZA
UNIVERSITÀ DI ROMA

Advanced Software Engineering

NNModelling

A Visual DSL for Neural Network Design

Authors

Davide Pietragalla 1965270

Luca Sforza 2050030

July 6, 2026

Contents

1	Introduction	3
2	Requirements	3
2.1	Functional Requirements	3
2.2	Non-Functional Requirements	4
3	Design	4
3.1	UML Metamodel	4
3.1.1	Design Principles and Requirement Satisfaction	6
3.2	Graph-to-NNTree Compilation Semantics	7
4	Static Tensor Type System	10
4.1	Tensor Types	10
4.2	Shape Dimensions	10
4.3	Typing Context	10
4.4	Typing Judgments	11
4.5	Inference Rules	11
4.6	Constraint-Based Inference Algorithm	12
4.7	Pattern Matching Semantics	12
5	Implementation	13
5.1	Front-End: Visual DSL Editor	13
5.1.1	Core Architecture	13
5.1.2	Node Rendering	13
5.1.3	NNTree Compiler Implementation	14
5.1.4	Connection Validation	14
5.2	Back-End: Network Instantiation	14
5.2.1	Configuration Generation	14
5.2.2	Dynamic Model Execution	15
5.2.3	Custom Operations	15
6	Conclusions	16
6.1	Achievements	16
6.2	Example: Transformer Classifier	17
6.3	Future Work	17
A	Transformer Architecture Diagram	18

1 Introduction

NNModelling is a DSL (Domain-Specific Language) for designing neural network architectures using a visual diagrammatic language. Models drawn in the visual editor are compiled into Hydra configuration files, which dynamically instantiate a working PyTorch Lightning model at runtime, enabling training and inference without manual implementation of the architecture.

The project follows a model-driven approach: the DSL syntax was first defined using a UML metamodel, then a front-end application was built to enable visual diagramming, and finally a back-end pipeline was implemented to produce executable configurations.

The system splits into two subsystems: a front-end visual editor built with Svelte 5, Svelte Flow, and TypeScript, where users drag, connect, and configure neural network modules; and a back-end pipeline written in Python using PyTorch Lightning and Hydra/OmegaConf, which converts visual diagrams into Hydra configuration files and dynamically instantiates them into a working PyTorch model.

The core data pipeline is:

Visual Graph $\rightarrow$ NNTree (Intermediate Representation) $\rightarrow$ JSON $\rightarrow$ Hydra YAML $\rightarrow$ LightningModule

Currently the system supports 27 modules (Linear, Conv2d, ReLU, Tanh, Embedding, LayerNorm, Dropout, various loss functions, etc.), 6 joins (Addition, Concat, Einsum, MatMul, ScaledDotProduct, MaskedScaledDotProduct), and 2 subflow behavioral templates (Repeat for sequential repetition and HorizontalRepeat for parallel copies via `vmap`).

2 Requirements

2.1 Functional Requirements

- **Visual Diagram Editor:** The system shall provide a drag-and-drop canvas where users can place, connect, and configure neural network module nodes using a visual interface.
- **Node Types:** The system shall support distinct node types including modules (single input, single output), input nodes (output only), loss/output nodes, joins (multiple inputs, single output), and subflow containers (collapsible nested graphs).
- **Implicit Forks:** Any node output may connect to multiple downstream nodes without requiring a special fork node.
- **Explicit Joins:** Joins shall be explicit special nodes that accept multiple inputs and produce a single output. Each join has an expression defining the merge operation (addition, concatenation, matrix multiplication, etc.).
- **Stereotype System:** The system shall support stereotype definitions stored as JSON files that define module behavior including Python class name, visual properties (color, dimensions), typed parameters with defaults, and category.
- **Extensible Module Registry:** Adding a new neural network layer shall require only a new JSON stereotype file, with no changes to the editor, compiler, or config generator.
- **Subflow Containers:** The system shall support hierarchical subflow containers that group related nodes into reusable sub-architectures.
- **Behavioral Stereotypes:** Subflows shall support behavioral stereotypes including **Repeat** (duplicates internal subgraph N times in sequence) and **HorizontalRepeat** (creates N parallel copies with independent weights executed via `vmap`).

- **Parameter Configuration:** Each node shall support typed parameters (int, float, bool, list, dict, None) configurable through the visual editor.
- **Graph Compilation:** The system shall transform a visual neural-network DAG into an execution-oriented hierarchical representation (NNTree) preserving graph semantics.
- **Config Generation:** The system shall generate executable Hydra configuration files from the compiled tree representation.
- **Dynamic Model Instantiation:** The back-end shall dynamically build a PyTorch Lightning module from the generated configuration at runtime.
- **Training Pipeline:** The system shall support training the generated model with configurable optimizer, trainer, dataset, and logging parameters.

2.2 Non-Functional Requirements

- **Separation of Concerns:** The front-end shall handle only DSL logic (visual editing, graph compilation, stereotype management). The back-end shall handle only network instantiation, training, and inference. The two subsystems communicate via a well-defined JSON interchange format.
- **Test Coverage:** The system shall maintain comprehensive test coverage across both front-end and back-end components.
- **Expressiveness:** The DSL shall be capable of expressing modern neural network architectures including transformers with multi-head attention, residual connections, and sequential repetition.
- **Execution Correctness:** The forward pass shall use BFS topological traversal with in-degree tracking to guarantee correct execution order through complex fork-join topologies.

3 Design

3.1 UML Metamodel

The abstract syntax of NNModelling is formally specified through a UML metamodel designed in Visual Paradigm. The metamodel defines the fundamental concepts of the DSL and the relationships among them. For clarity, it is presented through three complementary views: the node hierarchy, the connection model, and the stereotype system.

The first view (Figure 1) defines the modelling elements available to users when constructing a neural network architecture. The central concept is **Node**, an abstract element representing a computational unit within the graph. Specialized node types capture different architectural roles: **Module** represents standard neural-network layers, **Input** identifies graph entry points, **Join** merges multiple data flows, and **Subflow** groups nodes into hierarchical subgraphs.

A key design decision is the introduction of recursive subflows. Through the composition relationship between **Subflow** and **Node**, a subflow may contain any node type, including other subflows. This enables hierarchical modelling and allows complex architectures to be decomposed into reusable and manageable building blocks.

The second view (Figure 2) specifies how nodes are interconnected. Rather than connecting nodes directly, the metamodel introduces **Handle** elements as explicit connection endpoints. **SourceHandle** and **TargetHandle** specialize the generic **Handle** concept, while a **Connection** links a source handle to a target handle, forming a directed edge in the graph.

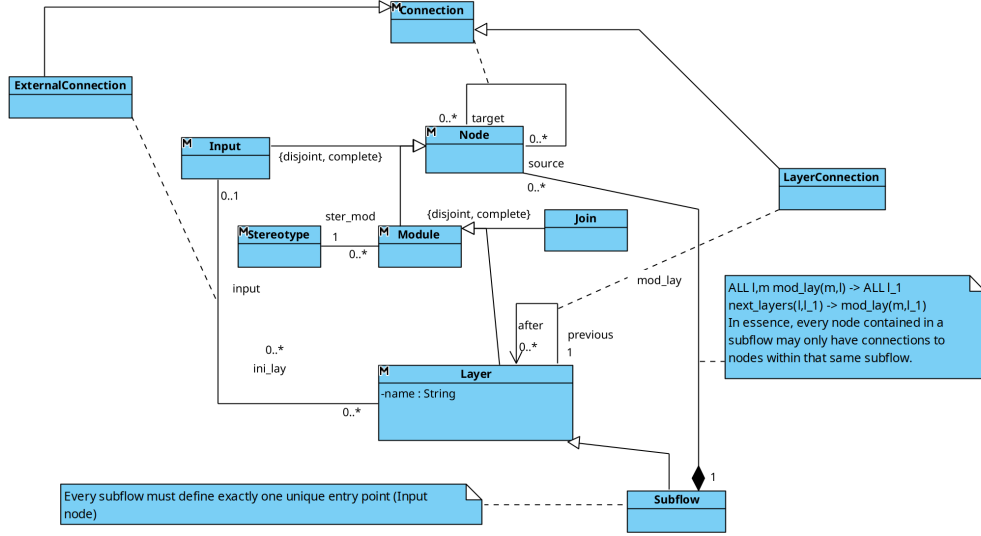


Figure 1: UML metamodel view showing the node hierarchy.

This design supports one of the key semantic choices of the DSL: implicit forks and explicit joins. A source handle may participate in multiple connections, allowing a node output to feed multiple downstream nodes without requiring dedicated fork elements. Conversely, merging multiple flows requires an explicit **Join** node, making data aggregation operations visible in the model.

The third view (Figure 3) defines the extensibility mechanism of the DSL. Instead of hard-coding neural-network layer types into the language, NNModelling uses stereotypes to describe module semantics, visual properties, and configurable parameters.

A **Stereotype** acts as a reusable layer definition and is associated with a set of **Parameters**. Concrete nodes do not reference stereotypes directly; instead, a **StereotypeApplication** associates a stereotype with a specific module instance and stores the corresponding **ParameterInstance** values. This separation between definition and instantiation allows the same layer type to be reused multiple times with different configurations.

Together, these three views define the abstract syntax of the DSL. The **node hierarchy** specifies the available modelling constructs, the **connection model** defines graph topology and data-flow semantics, and the **stereotype system** provides extensibility through configurable layer definitions. The implementation presented in the next section is directly derived from this metamodel.

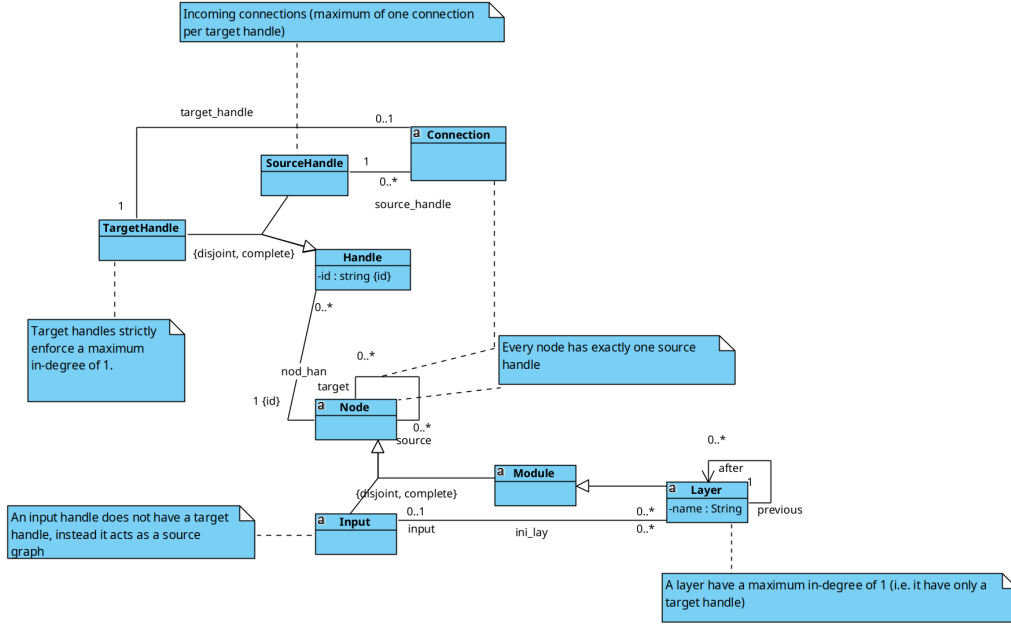


Figure 2: UML metamodel view showing the connection model.

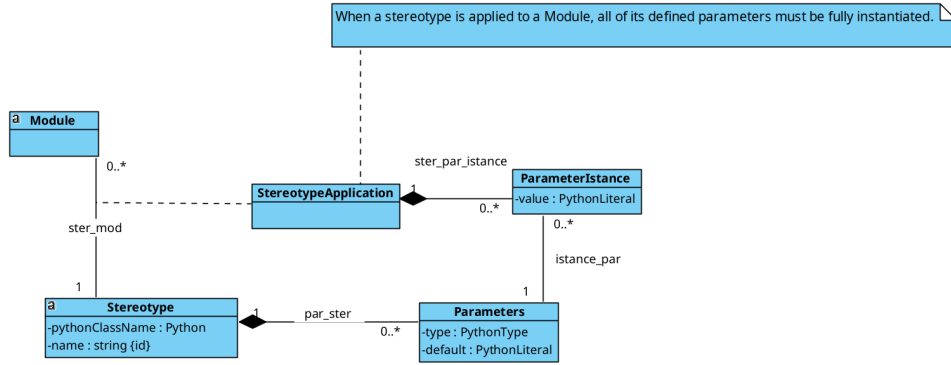


Figure 3: UML metamodel view showing the stereotype system.

3.1.1 Design Principles and Requirement Satisfaction

The metamodel was designed to satisfy the functional requirements identified in Section 2 while maintaining a compact and extensible DSL.

First, neural network architectures are represented as directed graphs, where nodes denote computational elements and edges represent tensor flow. This design directly addresses the requirement of modelling arbitrary network topologies, including sequential pipelines, branching structures, residual connections, and attention mechanisms. The adoption of implicit forks and explicit joins reduces visual complexity while preserving the semantics of data aggregation and multi-input operations.

Second, the metamodel supports extensibility through the stereotype mechanism. Rather than hardcoding layer types into the language, neural-network operations are described through configurable stereotypes containing semantic, visual, and parameter information. This design satisfies the requirement of supporting a wide range of neural-network components while enabling future extensions without modifications to the editor, compiler, or execution pipeline.

Third, hierarchical modelling is achieved through recursive subflows. By allowing subflows

to contain arbitrary nodes, including other subflows, the metamodel supports the construction of large architectures from reusable building blocks. This directly addresses the requirement of modelling complex architectures while maintaining readability and reducing diagram complexity.

Finally, the separation between stereotype definitions and their concrete applications enables the same neural-network component to be instantiated multiple times with different parameter values. Together with the graph-based representation and hierarchical subflows, this provides the flexibility required to model both simple feed-forward networks and more sophisticated architectures such as residual networks and transformers.

Overall, the metamodel serves as the formal foundation of the DSL, ensuring that the identified functional requirements are reflected directly in the language structure and can be consistently enforced throughout the implementation.

3.2 Graph-to-NNTree Compilation Semantics

The visual editor allows users to model neural network architectures as directed acyclic graphs (DAGs). While this representation is intuitive for design purposes, it is not directly suitable for runtime instantiation. The graph is therefore transformed into an intermediate hierarchical representation called *NNTree*, which captures the execution semantics of the DSL while remaining easily translatable into Hydra configuration files and PyTorch modules.

Formally, a model is represented as a directed acyclic graph

$$G = (V, E)$$

where V is the set of nodes and E is the set of directed connections.

Each node belongs to one of the DSL categories

$$type(v) \in \{Input, Module, Join, Subflow, Loss\}.$$

The compilation process transforms G into a hierarchical execution structure

$$T = (N, C)$$

where each element of N belongs to one of the following categories:

$$type(n) \in \{Sequential, Module, Join, Subflow\}.$$

The transformation preserves the execution semantics of the original graph while reducing structural complexity and making the architecture directly executable.

Sequential Compression A maximal linear path

$$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$$

such that

$$indegree(v_i) = 1 \quad \text{and} \quad outdegree(v_i) = 1$$

for every internal node is compressed into a single **Sequential** element.

The resulting node represents the function composition

$$f(x) = f_k \circ f_{k-1} \circ \dots \circ f_1(x),$$

which can be instantiated directly as a PyTorch `nn.Sequential` module.

Join Semantics Whenever a node receives multiple incoming connections

$$\text{indegree}(v) > 1,$$

the node is interpreted as a join operation.

A join represents a tensor merge operator

$$J : T_1 \times T_2 \times \dots \times T_n \rightarrow T,$$

where the concrete implementation depends on the selected stereotype (e.g. Addition, Concat, MatMul, Einsum, or ScaledDotProduct).

Input ordering is preserved through handle identifiers associated with incoming edges, ensuring deterministic execution.

Subflow Semantics A subflow represents a hierarchical graph encapsulation.

Formally, a subflow is itself a DAG

$$S = (V_S, E_S)$$

with

$$V_S \subseteq V.$$

The compilation process is recursively applied to every subflow, producing an independent NNTree representation for each nested graph.

This recursive definition enables arbitrary hierarchical structures while preserving the semantics of the original model.

Loss Nodes Loss nodes are treated as terminal elements of the graph.

Although represented visually as standard nodes, they are excluded from the execution graph and stored separately. Their purpose is to define the optimization objective

$$L(\hat{y}, y),$$

used during training.

Compilation Algorithm Compilation starts from the unique input node and recursively traverses the graph. The algorithm applies four transformations:

1. maximal linear chains are compressed into **Sequential** nodes;
2. merge points are converted into **Join** nodes;
3. subflows are recursively compiled;
4. terminal loss nodes are extracted and stored separately.

The resulting NNTree constitutes an execution-oriented representation of the DSL that is independent of any specific deep learning framework while remaining directly translatable into Hydra configurations and PyTorch modules.

Algorithm 1: Graph-to-NNTree Compilation

Input: Directed acyclic graph $G = (V, E)$

Output: Hierarchical execution tree T

Function *Compile*(v)

```
    if  $v$  is a Subflow then
        return CompileSubflow( $v$ )
    if  $\text{indegree}(v) > 1$  then
        return CreateJoin( $v$ )
    if  $v$  belongs to a maximal linear chain then
        return CompressSequential( $v$ )
    if  $v$  is a Loss node then
        StoreAsLoss( $v$ );
        return null
    return CreateModule( $v$ )
```

Function *CompileSubflow*(S)

```
    order  $\leftarrow$  TopologicalSort( $S$ );
    foreach  $v \in$  order do
        Compile( $v$ )
    return SubflowTree
```

$root \leftarrow$ unique Input node;

$T \leftarrow$ *Compile*($root$);

return T

4 Static Tensor Type System

A fundamental challenge in visual DSLs for neural network design is the lack of compile-time feedback about tensor shape compatibility. In NNModelling, a user may connect a **Linear** layer expecting 784 input features to a **Conv2d** layer producing a 3D tensor without any editor-level warning. The shape mismatch is only discovered at runtime when PyTorch raises a dimension error during training.

To address this, we introduce a **static tensor type system** capable of verifying tensor shapes and dtypes during visual editing and front-end compilation. The type system is integrated into the DSL as a separate verification pass, preserving the existing architecture while extending the language with compile-time safety guarantees.

4.1 Tensor Types

A tensor type τ is a pair consisting of a shape and a dtype:

$$\tau ::= \text{Tensor}(\sigma, \delta)$$

where σ is a tensor shape and δ is a tensor dtype (e.g., `float32`, `float64`, `int64`).

4.2 Shape Dimensions

A shape σ is a finite sequence of dimensions $d_1, d_2, \dots, d_n$. Each dimension d belongs to one of the following categories:

$$d ::= c \mid x \mid p \mid *$$

where:

- $c \in \mathbb{N}$ is a **constant** dimension (e.g., 3, 784, 1)
- $x \in \mathcal{X}$ is a **symbolic** dimension variable (e.g., B for batch size, C for channels, H for height, W for width) — these represent unknown dimensions whose values are determined during type inference but remain symbolic in the type representation
- $p \in \mathcal{P}$ is a **parameter reference** (e.g., `in_features`, `out_channels`) — these refer to node parameter values that are resolved at inference time
- $*$ is the **wildcard** dimension, matching zero or more arbitrary dimensions of any size. A wildcard in an input pattern consumes any matching dimensions from the actual tensor; in an output pattern, it reproduces the dimensions consumed during input matching

This representation allows the type system to express shapes that are partially known (containing symbolic variables and wildcards) rather than requiring every dimension to be a concrete integer. This is essential for modeling neural network architectures, where batch size is unknown at definition time and intermediate feature dimensions depend on upstream layers.

4.3 Typing Context

The typing context Γ is a partial mapping from symbolic dimension names to their resolved dimension values:

$$\Gamma ::= \{x_1 \mapsto d_1, x_2 \mapsto d_2, \dots\}$$

The context is populated incrementally during type inference as symbolic dimensions are bound to concrete values. Additionally, Γ carries the dtype environment and maintains a mapping from node identifiers to their inferred tensor types.

4.4 Typing Judgments

The central judgment form for node-level type inference is:

$$\Gamma, P \vdash M : (\tau_{\text{in}} \rightarrow \tau_{\text{out}})$$

meaning: “in context Γ with parameter values P , module M maps input type τ_{in} to output type τ_{out} .”

For graph-level inference, we extend the judgment to:

$$\Gamma \vdash G : \Gamma'$$

meaning: “graph G is well-typed, producing the extended environment Γ' containing type annotations for every node.”

4.5 Inference Rules

The typing rules are defined per module type. Each rule is derived declaratively from the `type_signature` field in the module’s stereotype JSON, rather than being hardcoded in the inference engine. This data-driven approach ensures that adding a new module requires only extending its stereotype definition, with no changes to the TypeScript implementation.

Input Node The Input node is a source in the computation graph. It produces a tensor whose last dimension is determined by its `out_features` parameter and whose batch dimension is a fresh symbolic variable:

$$\frac{\text{stereotype}(v) = \text{Input} \quad P = \text{params}(v)}{\Gamma \vdash v : \text{Tensor}((B, P.\text{out_features}), \text{float32})}$$

where B is a fresh symbolic dimension variable introduced into Γ .

Linear Layer A Linear layer with parameters P applies an affine transformation to the last dimension of its input. Given an input tensor of type $\text{Tensor}((B, \alpha_1, \dots, \alpha_k, F), \delta)$:

$$\frac{\Gamma \vdash x : \text{Tensor}((B, \alpha_1, \dots, \alpha_k, F), \delta) \quad F = P.\text{in_features}}{\Gamma \vdash \text{Linear}(P)(x) : \text{Tensor}((B, \alpha_1, \dots, \alpha_k, P.\text{out_features}), \delta)}$$

where $\alpha_1, \dots, \alpha_k$ are intermediate dimensions matched by a wildcard pattern and carried forward unchanged. The last dimension F must equal the declared `in_features` parameter; if this constraint is violated, a type error is emitted. The output preserves the batch dimension, all intermediate dimensions, and the dtype, while the last dimension becomes `out_features`.

ReLU Activation Activation functions are shape-preserving and dtype-preserving:

$$\frac{\Gamma \vdash x : \text{Tensor}(\sigma, \delta)}{\Gamma \vdash \text{ReLU}(x) : \text{Tensor}(\sigma, \delta)}$$

The same rule applies to all element-wise activation functions (Tanh, Sigmoid, Softmax) and normalization layers (BatchNorm, LayerNorm, Dropout), making their type signatures trivial to add.

4.6 Constraint-Based Inference Algorithm

Rather than implementing module-specific typing logic as TypeScript conditionals, the type engine implements a **constraint generation and constraint solving** approach. This design choice is fundamental to the extensibility of the type system.

The algorithm operates in two phases:

1. **Constraint Generation:** For each node visited in topological order, the engine reads the node’s stereotype `type_signature` and generates constraints by pattern-matching the actual input tensor shape against the declared input pattern. This produces:
 - **Bindings:** symbolic dimensions in the pattern are bound to their matched concrete values
 - **Substitutions:** wildcard dimensions capture a suffix of the actual shape for reuse in the output pattern
 - **Resolutions:** parameter references are resolved to the node’s current parameter values
2. **Constraint Solving:** The engine substitutes bound variables and captured wildcards into the output pattern, producing the output tensor type. If any constraint is violated (e.g., a constant dimension does not match, a dtype constraint fails, or a parameter reference cannot be resolved), a `TypeError` is recorded.

The inference is **forward-only** in Phase 1: types propagate from the Input node through the graph, and each node’s output type is computed before its successors are visited. Future phases will support backward inference (e.g., deducing input sizes from loss function requirements) and bidirectional constraint propagation (e.g., inferring `in_features` from context).

4.7 Pattern Matching Semantics

The pattern matching algorithm handles each dimension kind as follows:

- **Constant (c):** The actual input dimension at this position must equal c . If it does not, a type error is emitted (e.g., “dimension mismatch: expected 3, got 1”).
- **Symbolic (x):** If x is already bound in Γ , the actual dimension must equal the bound value (unification). If x is unbound, a new binding $x \mapsto d$ is added to Γ .
- **Parameter reference (p):** The parameter p is resolved from the node’s parameter map. The actual dimension must equal the resolved value.
- **Wildcard ($*$):** Consumes zero or more dimensions from the actual input shape. The consumed dimensions are captured and substituted into the corresponding position in the output pattern.

5 Implementation

The system is divided into two independent subsystems. The front-end contains all DSL logic (visual editing, graph compilation, stereotype management) and runs entirely in the browser. The back-end consumes the compiled JSON output and instantiates the corresponding PyTorch model. Currently, communication between the two subsystems occurs via file exchange (JSON export/import).

5.1 Front-End: Visual DSL Editor

The front-end is built with Svelte 5, Svelte Flow ([@xyflow/svelte](#)), and TypeScript. It runs entirely in the browser with no server-side dependencies.

5.1.1 Core Architecture

The central state manager is the `Diagram` class, implemented using Svelte 5 reactive `$state.raw` arrays. It stores the complete graph representation and exposes operations for manipulating nodes, edges, and subflows. The reactive programming model ensures that changes to the internal state are immediately propagated to the visual editor.

The stereotype mechanism provides the bridge between the UML metamodel and the executable DSL. Each stereotype is stored as a JSON artifact that instantiates the concepts defined in the metamodel, specifying visual attributes, semantic category, and configurable parameters. At startup, all stereotype definitions are discovered and loaded from the `Stereotypes/**/*.json` directory.

To simplify their use within the editor, the `Stereotype` class (`stereotype.ts`) wraps the raw JSON representation and exposes derived predicates such as `isJoin`, `isInput`, `isLoss`, and `isSubFlow`. This approach provides a type-safe interface and supports extensibility without requiring modifications to the compilation or rendering logic.

5.1.2 Node Rendering

Three Svelte components render different node types on the canvas:

- `CustomNode.svelte`: Standard module nodes with a single input handle on top and single output handle on bottom. Visual appearance (color, size) is driven by the stereotype's `view` properties. Parameters are displayed according to their declared positions (top, bottom, or inline).
- `JoinNode.svelte`: Multi-input merge nodes with N input handles (dynamically generated based on incoming edge count) and a single output handle. The node label shows the join operation type.
- `SubflowNode.svelte`: Collapsible container nodes with a colored header bar. When collapsed, internal nodes are hidden but still participate in compilation. When expanded, internal nodes are rendered as children of the subflow group.

5.1.3 NNTree Compiler Implementation

The graph-to-NNTree compilation semantics described in Section 3.1.2 are implemented by the `NNTree` class located in

`frontend/src/lib/conversion/nntree.ts`

The compiler operates directly on the visual graph stored by the `Diagram` class and produces a hierarchical intermediate representation that can be serialized as JSON and consumed by the back-end pipeline.

In model-driven engineering terms, the compiler realizes the model-to-model transformation

$$G = (V, E) \rightarrow T = (N, C)$$

defined by the DSL semantics. The resulting NNTree serves as the executable intermediate representation used by all subsequent stages of the pipeline.

Compilation starts from the unique Input node and performs a recursive traversal of the graph. The implementation closely follows Algorithm 1. During traversal, the compiler identifies the structural patterns defined by the DSL semantics and creates the corresponding NNTree nodes.

Linear chains are detected and compressed into `Sequential` nodes through the `createSequential()` routine. Join nodes are recognized whenever a node has multiple incoming edges and are processed through `handleJoin()`. Subflows are recursively compiled through `processSubflow()` and `compileSubflowGraph()`, which build independent hierarchical representations for nested graphs.

The implementation uses Kahn’s topological sorting algorithm to validate and order the internal contents of subflows before compilation. Cyclic subflow definitions are rejected and reported as errors, ensuring that all compiled structures remain valid DAGs.

The resulting NNTree structure is stored as a collection of `NNTreeNode` objects and can be exported through the `toJson()` method. This serialized representation forms the interface between the front-end DSL environment and the back-end execution pipeline.

5.1.4 Connection Validation

The `checkValidConnection` function enforces connection rules: each target handle accepts only one connection; source handles allow unlimited outgoing connections (implicit forks); ancestry loop detection prevents circular parent references; and nodes dragged into subflow containers are auto-reparented.

5.2 Back-End: Network Instantiation

The back-end is written in Python and is responsible for consuming the compiled NNTree JSON, generating Hydra configuration files, and dynamically instantiating a working PyTorch Lightning model. It uses PyTorch for tensor operations, PyTorch Lightning for training infrastructure (LightningModule, Trainer), and Hydra/OmegaConf for configuration management.

5.2.1 Configuration Generation

The pipeline script reads the NNTree JSON and generates seven YAML configuration files:

- `net/`: Network architecture with node definitions, layer targets, and parameter values.
- `optimizer/`: Optimizer configuration (Adam, SGD, etc.) with learning rate and hyper-parameters.

- `trainer/`: Lightning Trainer settings (max epochs, accelerator, precision).
- `dataset/`: Dataset class selection and parameters.
- `wandb/`: Weights & Biases logging configuration.
- `early_stopping/`: Early stopping criteria and patience.

Key processing steps include: parsing parameter strings via `ast.literal_eval` (supporting `int`, `float`, `bool`, `list`, `dict`, and `None` types); resolving `pythonClassName` to Hydra `_target_` paths (e.g., `nn.Linear` $\rightarrow$ `torch.nn.Linear`); using `_recursive_`: `False` on subflow nodes to prevent Hydra from recursing into internal node maps; detecting task type (classification/regression) from loss function selection for automatic metric configuration; and supporting `--num-classes` and `--dataset` CLI flags for configurable output dimensions and dataset selection.

5.2.2 Dynamic Model Execution

The `Net` class, a `LightningModule`, dynamically builds its architecture from the Hydra configuration at runtime:

- A `ModuleDict` is populated by dispatching on node type: `sequential` chains are wrapped in `nn.Sequential`; `module` nodes are instantiated individually via Hydra’s `instantiate`; `join` nodes reference custom operation classes; and `subflow` nodes reference `Subflow`, `Repeat`, or `HorizontalRepeat` operation classes.
- The forward pass uses BFS traversal with in-degree tracking to ensure correct execution order through complex fork-join topologies.
- Join nodes collect all inputs ordered by `targetHandle` before executing the merge operation.
- Loss functions are excluded from the execution graph and stored separately for metric selection. Although the DSL models loss nodes as regular layers with a scalar tensor output, the runtime does not wire this value into the optimizer — loss nodes remain terminal.

5.2.3 Custom Operations

Eleven custom Python operations are implemented in the back-end, spanning three categories:

- **Join operations (6)**: `Addition`, `Concat`, `Einsum`, `MatMul`, `ScaledDotProduct`, and `MaskedScaledDotProduct` handle multi-input merges. Each inherits from a common base and implements a `forward` method that receives an ordered list of input tensors.
- **Subflow operations (3)**: `Subflow` implements BFS internal graph traversal for arbitrary subgraphs; `Repeat` wraps `N` sequential copies in `nn.Sequential`; `HorizontalRepeat` uses `vmap` with `functional.call` and `stack_module_state` for `N` parallel copies. `HorizontalRepeat`’s join is currently hardcoded to concatenation on the last dimension.
- **Module operations (2)**: `PositionalEncoding` computes sinusoidal positional encodings; `SequencePool` computes mean pooling over the sequence dimension.

6 Conclusions

6.1 Achievements

NNModelling demonstrates that a visual DSL can effectively bridge the gap between neural network architecture design and implementation. The system successfully compiles complex architectures including transformers with multi-head attention, residual connections, nested subflows, and sequential repetition.

Key achievements include:

- A full end-to-end pipeline from visual design to trained model.
- An extensible stereotype system requiring no code changes for new layer types — 27 modules, 6 joins, and 2 subflow templates are currently supported.
- Support for complex architectural patterns: subflows with recursive nesting, residual connections, parallel branches via `vmap`-based `HorizontalRepeat`, and sequential repetition via `Repeat`.
- Comprehensive test coverage (179 tests) across both front-end and back-end, ensuring reliability of the compilation and config generation pipeline.
- A model-driven approach with a formal UML metamodel, ensuring the DSL syntax is well-defined and verifiable.
- Clean separation of concerns: the front-end handles all DSL logic (visual editing, graph compilation, stereotype management), while the back-end handles network instantiation, training, and inference. This decoupling enables independent development and testing of each subsystem.
- A dynamic model instantiation system that builds PyTorch Lightning modules from configuration at runtime, enabling rapid experimentation without code changes.

6.2 Example: Transformer Classifier

To illustrate a complete DSL model in practice, the transformer classifier architecture (shown in Appendix A, Figure 5) defines a hierarchical DAG spanning three abstraction levels: a top-level pipeline (Input $\rightarrow$ Embedding $\rightarrow$ PositionalEncoding $\rightarrow$ TransformerEncoder $\rightarrow$ SequencePool $\rightarrow$ Linear $\rightarrow$ CrossEntropyLoss), a **Repeat** encoder subflow with residual blocks and FFN, and a **HorizontalRepeat** multi-head attention subflow with Q/K/V projections and scaled dot-product.

This transformer is trained on the EnronSpam dataset to classify emails as Spam or Ham. On the test set, the model correctly classifies 98.4% of emails.

This experiment demonstrates that the DSL is expressive enough to model a non-trivial modern architecture involving hierarchical subflows, residual connections, attention mechanisms, and repeated structures.



Figure 4: Training metrics for the transformer classifier on the EnronSpam dataset.

6.3 Future Work

Several directions for future development are planned:

- **Network Communication:** The current file-based exchange between front-end and back-end will be replaced with a network communication layer. A training server API is needed to enable one-click training directly from the visual editor, with the front-end sending diagrams to the back-end over HTTP or WebSocket. This will eliminate the manual file export-import workflow and enable real-time training progress monitoring from the editor.
- **Configurable Join Type:** The HorizontalRepeat’s hardcoded concatenation join will be replaced with a configurable `join_type` parameter supporting sum, mean, product, or custom aggregation strategies.
- **Parallel Graph Execution:** The single-threaded topological iteration can be replaced with a concurrent CUDA stream dispatcher for arbitrary fork-join topologies, enabling true parallel execution of independent branches on separate GPU streams.
- **Subflow Persistence:** Users shall be able to save and load individual subflows to and from disk independently.
- **Interactive Deployment:** HuggingFace Spaces deployment is planned for interactive browser demos, allowing users to test NNModelling without local installation.
- **Pre-trained Model Support:** Integration of transfer learning workflows with support for loading pre-trained weights into DSL-defined architectures, enabling fine-tuning scenarios.

A Transformer Architecture Diagram

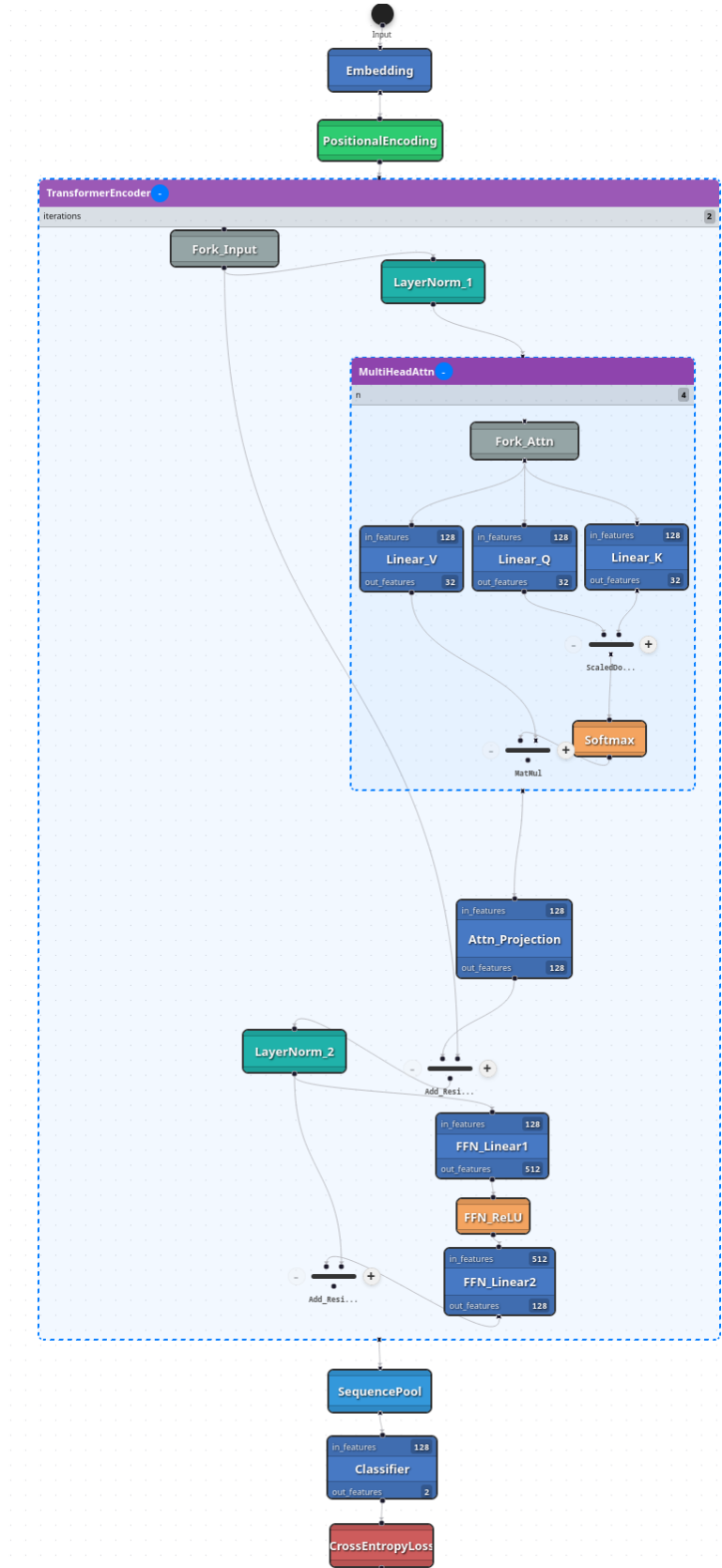


Figure 5: Full transformer classifier graph as rendered in the NNModelling visual editor.

References

- [1] Svelte 5 Documentation. <https://svelte.dev/docs>
- [2] Svelte Flow Documentation. <https://svelteflow.dev/>
- [3] Paszke, A. et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” NeurIPS 2019.
- [4] Falcon, W. et al. “PyTorch Lightning.” <https://lightning.ai/>
- [5] Vaswani, A. et al. “Attention Is All You Need.” NeurIPS 2017.
- [6] Yadan, O. “Hydra - A Framework for Elegantly Configuring Complex Applications.” <https://hydra.cc/>